

VirtuEx 시스템 감사 보고서 (14차)

VirtuEx System Audit Report (14th)

- 감사일: 2026-03-13
- 감사 범위: 프론트엔드/백엔드 전체 코드 리팩토링, 주석, 성능, UX, 로깅, 문서, 설정 파일 종합 감사
- 감사 방법: 전체 소스 코드 정적 분석 + 설정 파일 교차 검증 + 문서 일관성 점검
- 심각도 기준: Critical(즉시 수정) / High(1주 내) / Medium(2주 내) / Low(개선 권장)
- 비고: 본 보고서는 발견 사항만 기록하며, 수정은 포함하지 않음

요약 (Executive Summary)

A. 프론트엔드 코드 품질 — 5건

[FE-H-01] RichEditor 하드코딩된 alert 메시지 (High)

- 파일: frontend/src/components/ui/RichEditor.tsx:90, 95
- 설명: 이미지 업로드 실패 시 `alert()` 호출에 한/영 텍스트가 하드코딩되어 있음. i18n 시스템(`t()` 함수) 미사용. 또한 `alert()` 은 브라우저 기본 다이얼로그로 앱의 토스트 UI 패턴과 불일치
- 권장: i18n 키 등록 + toast 알림으로 변경

[FE-M-01] TipTap 에디터 정적 임포트 (Medium)

- 파일: frontend/src/components/ui/RichEditor.tsx
- 설명: TipTap 에디터(@tiptap/react, @tiptap/starter-kit 등)가 정적 임포트됨. 번들 사이즈 약 150KB+. 커뮤니티 글 작성 페이지에서만 사용되므로 `next/dynamic` 으로 지연 로드 가능
- 권장: `dynamic(() => import('./RichEditor'), { ssr: false })` 로 변경

[FE-M-02] 일부 라우트 error.tsx 누락 (Medium)

- 파일: `frontend/src/app/(main)/community/[id]/`, `app/(main)/orders/`, `app/(main)/mypage/edit/` 등
- 설명: 메인 라우트(`/`, `/dashboard`, `/login`)에는 `error.tsx`가 있으나, 일부 하위 라우트에는 `error.tsx`가 없어 에러 발생 시 상위 경계로 버블업됨. 사용자에게 해당 페이지 맥락에 맞는 에러 메시지를 보여주기 어려움
- 권장: 주요 동적 라우트에 `error.tsx` 추가

[FE-L-01] Header 언어 토글 하드코딩 (Low)

- 파일: `frontend/src/components/layout/Header.tsx:334, 463`
- 설명: `{locale === 'ko' ? 'English' : '한국어'}` — 언어 전환 버튼 텍스트가 i18n 시스템 외부에 하드코딩. 기능상 문제는 없으나 일관성 부족
- 권장: i18n 키 등록 또는 현재 패턴 유지 (의도적 하드코딩으로 볼 수도 있음)

[FE-L-02] auth 스토어 보안 TODO 미해결 (Low)

- 파일: `frontend/src/stores/auth.ts:37-38`
- 설명: TODO: `HttpOnly + Secure + SameSite` 쿠키로 토큰 저장 전환 — XSS 완전 차단 주석이 남아있음. `localStorage`에 JWT 저장 중으로 XSS 공격 시 토큰 탈취 가능
- 권장: `HttpOnly` 쿠키 전환 검토 (프로덕션 배포 전)

B. 백엔드 코드 품질 — 7건

[BE-H-01] 테스트 파일 `as any` 타입 단언 다수 (High)

- 파일:
 - `portfolio/src/domain/services/watchlist.service.spec.ts:16` — `mockPrisma as any`
 - `order-engine/src/domain/services/matching-engine.service.spec.ts:8` — `null as any`
 - `order-engine/src/domain/aggregates/order.aggregate.spec.ts:318, 343` — `OrderAggregate as any`
 - `chat/src/chat/chat.service.spec.ts:66, 75, 95, 117` — `type 필드 as any`
- 설명: 테스트 파일에서 mock 객체 생성 시 `as any` 남용. 타입 안전성 상실로 런타임 에러를 컴파일 단계에서 감지 불가. 테스트가 실제 인터페이스 변경을 감지하지 못할 위험
- 권장: `Partial` 또는 `jest.Mocked` 패턴 사용

[BE-H-02] `balance.service.ts` `error as any` 타입 단언 (High)

- 파일: `portfolio/src/domain/services/balance.service.ts:146`
- 설명: `catch` 블록에서 `error as any` 사용. `instanceof Error` 체크 없이 직접 속성 접근하여 런타임 에러 가능
- 권장: `if (error instanceof Error) { ... }` 패턴으로 변경

[BE-M-01] DTO 파일 이중 주석 누락 (Medium)

- 파일:

- `portfolio/src/presentation/dto/deposit.dto.ts`
- `portfolio/src/presentation/dto/withdraw.dto.ts`
- `portfolio/src/presentation/dto/internal.dto.ts`
- `portfolio/src/presentation/dto/copy-trade.dto.ts`
- `order-engine/src/presentation/dto/` (일부)

- 설명: 최근 추가된 DTO 파일들에 한/영 이중 주석이 누락됨. 프로젝트 전반에 이중 주석 컨벤션이 적용되어 있으나 해당 파일들은 미적용

- 권장: 프로젝트 주석 컨벤션에 맞게 이중 주석 추가

[BE-M-02] copy-trade.service.ts 미완성 TODO (Medium)

- 파일: `portfolio/src/domain/services/copy-trade.service.ts:30-32`

- 설명: TODO: Move to config/env variable for per-environment tuning — 슬라이피지 허용치, 최대 깊이 등 상수가 코드에 하드코딩. 환경별 설정 분리 미완료

- 권장: ConfigService를 통한 환경변수 분리

[BE-M-03] binance-price.service.ts 메서드별 문서화 부족 (Medium)

- 파일: `market-data/src/domain/services/binance-price.service.ts`

- 설명: 대규모 서비스 파일(WebSocket 연결, 가격 파싱, 재연결 로직)임에도 메서드별 JSDoc 주석이 부족. 이중 주석 미적용 구간 존재

- 권장: 주요 메서드에 이중 주석 추가

[BE-L-01] health.controller.ts 빈 catch 블록 (Low)

- 파일: `api-gateway/src/health/health.controller.ts:147`

- 설명: `catch { /* stats are optional */ }` — 에러를 완전히 무시. 통계 수집 실패가 반복되어도 인지 불가

- 권장: 최소한 debug 레벨 로깅 추가

[BE-L-02] price-subscriber.service.ts 가격 알림 실패 무시 (Low)

- 파일: `api-gateway/src/gateway/price-subscriber.service.ts:250`

- 설명: `.catch((e) => this.logger.warn(...))` — 알림 연속화 실패를 warn 로깅만 하고 재시도 없음. 알림 유실 가능

- 권장: 실패 알림 큐 또는 재시도 메커니즘 검토

C. 보안 — 4건

[SEC-C-01] SMS 재전송/비밀번호 복구 엔드포인트 레이트 리미팅 누락 (Critical)

- 파일: `api-gateway/src/proxy/auth-proxy.controller.ts:106, 260, 276`
- 설명: `/login/resend-sms`, `/forgot-password/resend-sms`, `/forgot-password/verify-sms` 엔드포인트에 `@Throttle()` 데코레이터 미적용. 글로벌 `ThrottlerGuard`(60초/100회)는 있으나 해당 엔드포인트는 SMS 비용 발생 경로이므로 더 엄격한 개별 제한 필요. 자동화 공격으로 SMS 비용 폭증 가능
- 권장: `@Throttle({ default: { limit: 3, ttl: 60000 } })` 등 개별 제한 적용

[SEC-H-01] TOTP 엔드포인트 레이트 리미팅 누락 (High)

- 파일: `api-gateway/src/proxy/auth-proxy.controller.ts:331, 347, 364, 381`
- 설명: TOTP `setup/enable/disable/verify` 4개 엔드포인트에 개별 레이트 리미팅 없음. 6자리 TOTP 코드 브루트포스 공격 가능 (100만 조합, 글로벌 제한 60초/100회로는 부족)
- 권장: TOTP `verify`에 `@Throttle({ default: { limit: 5, ttl: 60000 } })` 적용

[SEC-H-02] 로그인 실패 시 이메일 주소 로깅 (High)

- 파일: `user-auth/src/application/services/auth.service.ts:258`
- 설명: `this.logger.warn('Login failed (invalid password) for: ${user.email}')` — 실패한 로그인 시도의 이메일 주소를 그대로 로그에 기록. 로그 유출 시 사용자 이메일 대량 노출 위험
- 권장: 이메일 마스킹 적용 (예: `d***@example.com`)

[SEC-M-01] SMS 인증 코드 개발환경 로깅 (Medium)

- 파일: `user-auth/src/application/services/sms-verification.service.ts:59-61`
- 설명: 개발 환경에서 SMS 인증 코드를 콘솔에 전체 출력. `NODE_ENV` 체크가 있으나, staging 환경에서도 노출될 가능성 존재
- 권장: `development` 환경만 출력하도록 엄격 체크 확인

D. 성능 — 5건

[PERF-H-01] 리더보드 전체 계좌 메모리 로드 (High)

- 파일: `portfolio/src/domain/services/balance.service.ts:841-862`
- 설명: `getLeaderboard()` 메서드에서 `prisma.account.findMany()` 로 모든 계좌를 메모리에 로드한 후 정렬. 사용자 10,000명 이상 시 메모리 과다 사용 + 응답 지연. 현재는 사용자 수가 적어 문제 없으나 확장성 부족

- 권장: SQL 레벨에서 ORDER BY + LIMIT 적용, 또는 집계 쿼리 사용

[PERF-M-01] 관리자 통계 count 쿼리 비효율 (Medium)

- 파일: `user-auth/src/presentation/controllers/statistics.controller.ts:75-82`
- 설명: 여러 모델에 대한 `count()` 쿼리를 개별 실행. `Promise.all` 로 병렬화되어 있으나, 일부 count는 필터 없이 전체 테이블 스캔 발생 가능
- 권장: 자주 조회되는 통계는 캐시 또는 집계 테이블 활용

[PERF-M-02] 채팅 참가자 목록 무제한 조회 (Medium)

- 파일: `chat/src/chat/chat.service.ts:27-53`
- 설명: `getRooms()` 호출 시 각 방의 전체 참가자 목록과 메시지를 제한 없이 조회. 대규모 그룹 채팅 시 응답 크기 증가
- 권장: 참가자 수 제한(예: 상위 50명) 또는 별도 API로 분리

[PERF-M-03] 가격 알림 심볼별 전체 로드 (Medium)

- 파일: `api-gateway/src/gateway/price-subscriber.service.ts:97-109`
- 설명: 알림 갱신 시 심볼별 모든 활성 알림을 매번 로드. 시간 기반 필터링 없음. 알림 수 증가 시 성능 저하
- 권장: 최근 변경된 알림만 로드하도록 변경

[PERF-L-01] 관리자 사용자 조회 2건 중복 쿼리 (Low)

- 파일: `user-auth/src/application/services/admin.service.ts:140-156`
- 설명: `approvedBy`, `rejectedBy` UUID를 사용자명으로 변환할 때 개별 쿼리 2회 실행. 배치 조회로 통합 가능
- 권장: `findMany({ where: { id: { in: [...] } } })` 사용

E. 로깅 — 5건

[LOG-H-01] WebSocket 메시지 파싱 실패 무시 (High)

- 파일: `market-data/src/domain/services/binance-price.service.ts:112-114`
- 설명: Binance WebSocket 메시지 JSON 파싱 실패를 `catch { }` 로 완전히 무시. 데이터 형식 변경 시 인지 불가. 잘못된 가격 데이터가 무시되면서 가격 갱신이 멈출 수 있음
- 권장: parse 실패 횟수 카운터 + 임계값 초과 시 경고 로깅

[LOG-M-01] 주문 에러 로그 컨텍스트 부족 (Medium)

- 파일: `order-engine/src/application/services/order.service.ts:1114, 1167, 1216, 1241`
- 설명: 에러 로그에 주문 ID, 거래 ID 등 상관관계 식별자가 누락. 장애 발생 시 특정 주문 추적 어려움
- 권장: 에러 로그에 `orderId`, `tradeId`, `symbol` 등 컨텍스트 추가

[LOG-M-02] 프록시 에러 로그 요청 정보 누락 (Medium)

- 파일: `api-gateway/src/proxy/proxy.service.ts:133`
- 설명: 프록시 전달 실패 시 요청 `method/URL`이 로그에 포함되지 않아 어떤 API 호출이 실패했는지 파악 어려움
- 권장: 에러 로그에 `method`, `url`, `targetService` 포함

[LOG-L-01] 자산 갱신 빈도 높은 정보 로그 (Low)

- 파일: `market-data/src/application/services/market-data.service.ts:100`
- 설명: 5분마다 자산 갱신 결과를 INFO 레벨로 로깅. 변경 없을 때도 `Refreshed: 0 added, 0 removed` 출력. 프로덕션에서 불필요한 로그 양 증가
- 권장: 변경 있을 때만 INFO, 없으면 DEBUG

[LOG-L-02] 정산 복구 재시도 로그 레벨 부적절 (Low)

- 파일: `order-engine/src/application/services/settlement-recovery.service.ts:105`
- 설명: 일시적 실패에 대한 재시도를 WARN 레벨로 로깅. 정상 복구 흐름에서도 WARN이 누적되어 모니터링 노이즈 발생
- 권장: 첫 재시도는 DEBUG, 3회 이상 실패 시 WARN

F. UX/접근성 — 3건

[UX-M-01] MarketIndexSummary 차트 접근성 부족 (Medium)

- 파일: `frontend/src/components/market/MarketIndexSummary.tsx:89-94`
- 설명: Canvas 기반 스파크라인 차트에 `aria-label` 이 없음. 스크린 리더 사용자가 차트 내용을 인식할 수 없음
- 권장: `aria-label` 에 지수명 + 변동률 정보 제공

[UX-L-01] 커뮤니티 에러 시 맥락 정보 부족 (Low)

- 파일: `frontend/src/app/(main)/community/[id]/page.tsx`

- **설명:** 게시글 로드 실패 시 상위 error.tsx로 버블업되어 "대시보드로 이동" 안내만 표시. "게시글을 찾을 수 없습니다" 등 맥락에 맞는 에러 메시지 미제공
- **권장:** 해당 라우트에 error.tsx 추가

[UX-L-02] 주문 내역 빈 상태 안내 부족 (Low)

- **파일:** frontend/src/app/(main)/orders/page.tsx
- **설명:** 주문 내역이 없을 때 "주문이 없습니다" 텍스트만 표시. 처음 사용하는 사용자에게 주문 방법 안내(CTA 버튼 등)가 없음
- **권장:** "첫 거래 시작하기" 등 CTA 추가 검토

G. 문서/설정 — 8건

[DOC-C-01] LOCAL_RUN.md Node.js 버전 불일치 (Critical)

- **파일:**
 - docs/LOCAL_RUN.md:12 — Node.js 20+ 요구
 - frontend/public/docs/LOCAL_RUN.md:12 — Node.js 18+ 요구
 - README.md:93 — Node.js >= 20 요구
- **설명:** 3개 문서에서 Node.js 최소 버전이 서로 다름. frontend/public/docs/LOCAL_RUN.md 는 내용이 40줄 짧아 구 버전으로 추정. 신규 개발자 온보딩 혼란 유발
- **권장:** frontend/public/docs/LOCAL_RUN.md 를 docs/LOCAL_RUN.md 와 동기화, 버전 20+로 통일

[DOC-H-01] 프론트엔드 README 부재 (High)

- **파일:** frontend/README.md — 파일 없음
- **설명:** 프론트엔드 디렉토리에 README가 없음. Next.js 프로젝트 구조, 개발 서버 실행법, 디렉토리 구조 설명 부재
- **권장:** 프론트엔드 구조, 주요 페이지, 개발 가이드를 포함한 README 작성

[DOC-H-02] 백엔드 서비스별 README 부재 (High)

- **파일:** backend/services/*/README.md — 8개 서비스 모두 없음
- **설명:** 각 마이크로서비스의 역할, API 목록, 환경변수, 의존관계를 설명하는 문서 없음. 루트 README에 개요는 있으나 개별 서비스 상세 정보 부족
- **권장:** 서비스별 최소 README 작성 (역할, 포트, 주요 API, 환경변수)

[DOC-H-03] package.json description 필드 누락 (High)

- **파일:**

- `frontend/package.json` — description 없음
- `backend/services/*/package.json` — 8개 서비스 모두 description 없음
- 설명: npm registry 등록 시 프로젝트 식별 불가. `npm ls` 등 도구에서 서비스 구분 어려움
- 권장: 각 `package.json`에 간략한 description 추가

[DOC-M-01] TypeScript 타겟 불일치 (Medium)

- 파일:
- `tsconfig.base.json:2` — `"target": "ES2022"`
- `frontend/tsconfig.json:3` — `"target": "ES2017"`
- 설명: 프론트엔드가 ES2017을 타겟으로 하여 백엔드(ES2022)와 불일치. Next.js가 자체 트랜스파일링을 하므로 실질적 영향은 적으나 프로젝트 일관성 부족
- 권장: Next.js의 기본 tsconfig 설정에 맞게 유지하되, 차이를 문서화

[DOC-M-02] .env NOTIFICATION_DATABASE_URL 미사용 의심 (Medium)

- 파일: `.env` — `NOTIFICATION_DATABASE_URL` 정의됨
- 설명: Notification 서비스에 Prisma 스키마가 없음. 해당 DB URL이 실제로 사용되는지 불명확. 불필요한 환경변수일 가능성
- 권장: 실제 사용 여부 확인 후 미사용 시 제거

[DOC-L-01] docs/api/ 디렉토리 빈 상태 (Low)

- 파일: `docs/api/` — 비어있음
- 설명: API 문서 디렉토리가 존재하나 내용 없음. Swagger가 개발 환경에서 제공되므로 큰 문제는 아니나 정적 API 문서가 없음
- 권장: Swagger 문서 참조 안내 추가 또는 디렉토리 제거

[DOC-L-02] 서킷 브레이커 상태 비영속 (Low)

- 파일: `api-gateway/src/proxy/proxy.service.ts`
- 설명: API Gateway 서킷 브레이커 상태가 인메모리에만 존재. 서버 재시작 시 초기화됨. 현재 로컬 개발 환경에서는 문제없으나 프로덕션 배포 시 이슈 가능
- 권장: 프로덕션 배포 시 Redis 기반 상태 공유 검토

검증 환경

이전 감사 대비 개선 현황

카테고리	Critical	High	Medium	Low	합계
프론트엔드 코드 품질	0	1	2	2	5
백엔드 코드 품질	0	2	3	2	7
보안	1	2	1	0	4
성능	0	1	3	1	5
로깅	0	1	2	2	5
UX/접근성	0	0	1	2	3
문서/설정	1	3	2	2	8
합계	2	10	14	11	37
항목	상태				
프론트엔드 빌드 검증					
백엔드 전체 타입 검사					
서비스 기동 확인					
문서 교차 검증					
차수	발견 건수	수정 건수	미수정	신규	
13차	8	8	0	-	
성능 1차	12	12	0	-	

14차37--37

- 13차 및 성능 1차 감사에서 발견된 모든 항목은 수정 완료됨
- 14차 감사는 수정 없이 발견 사항만 기록 (사용자 요청에 의함)
- 프론트엔드/백엔드 코드 품질, 보안, 성능, 로깅, UX, 문서 전 영역 종합 감사 완료